

Hotel Management System

Database Design Document

Entity-relationship model, schema definitions, and data integrity rules

Prepared for: [Client / Hotel Group Name]

Prepared by: Disura

Date: July 2026

Version: 1.0

Status: Draft for client review

Revision History

Version	Date	Author	Description
0.1	22 Jun 2026	Disura	Initial schema draft
1.0	01 Jul 2026	Disura	Complete database design issued for review

Table of Contents

1. Introduction

1.1 Purpose

This document describes the database design for the Hotel Management System: the entity-relationship model, table definitions, keys, constraints, and indexing strategy. It is intended to guide implementation and to serve as a reference for future maintenance.

1.2 Database Management System

PostgreSQL is used as the primary relational database. It was selected over alternatives such as MySQL or MongoDB primarily because of its native support for range types and exclusion constraints, which allow the system to guarantee – at the database level – that no two reservations can ever overlap for the same room, and its strong general-purpose transactional integrity, which matters when a single action (e.g., checkout) must atomically update room status, folio balance, and payment records together.

2. Entity-Relationship Diagram

The diagram below shows the core entities of the system and their relationships. Supporting/lookup tables (roles, permissions, audit log) are described in the table definitions but omitted from the diagram for clarity.

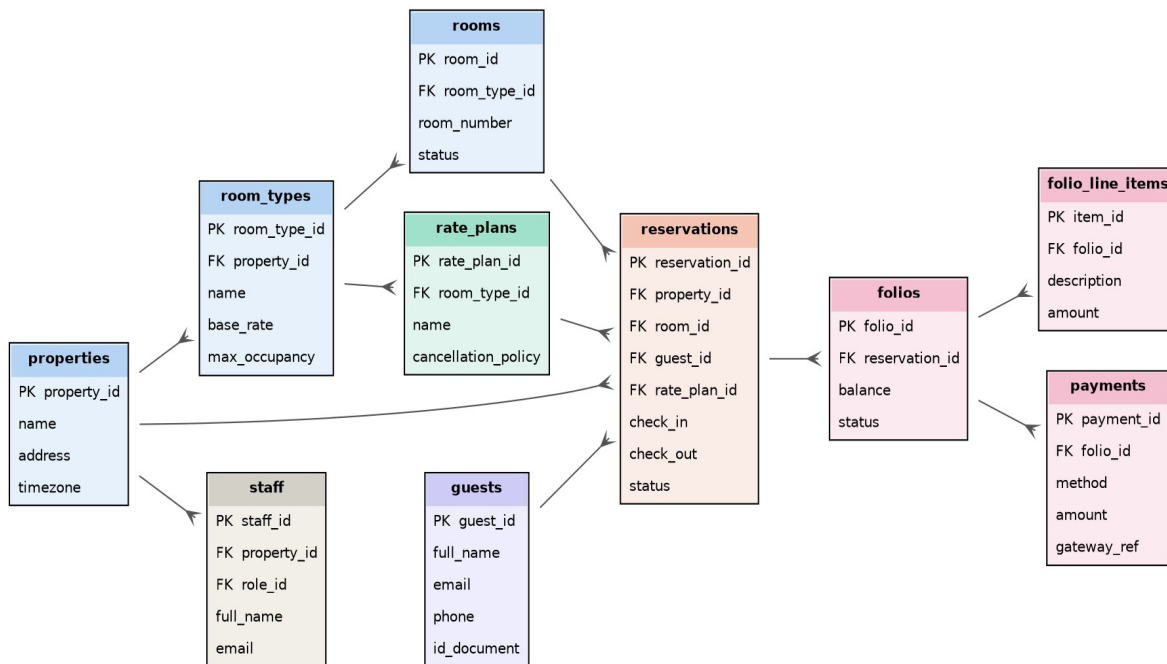


Figure 1: Entity-relationship diagram – core hotel management entities

3. Table Definitions

3.1 properties

Represents a single hotel/branch. All operational data is scoped to a property.

Column	Type	Constraints	Description
property_id	UUID	PK	Unique identifier
name	VARCHAR(150)	NOT NULL	Property/branch name
address	TEXT	NOT NULL	Physical address
timezone	VARCHAR(50)	NOT NULL, default 'Asia/Colombo'	IANA timezone used for all date/time calculations at this property
currency	CHAR(3)	NOT NULL	ISO 4217 currency code, e.g. LKR, USD
created_at	TIMESTAMPTZ	NOT NULL, default now()	Record creation time

3.2 room_types

Column	Type	Constraints	Description
room_type_id	UUID	PK	Unique identifier
property_id	UUID	FK → properties.property_id, NOT NULL	Owning property
name	VARCHAR(100)	NOT NULL	e.g. Deluxe Double, Family Suite
base_rate	NUMERIC(10,2)	NOT NULL, CHECK (base_rate >= 0)	Default nightly rate before rate-plan overrides
max_occupancy	SMALLINT	NOT NULL, CHECK (max_occupancy > 0)	Maximum guests permitted

3.3 rooms

Column	Type	Constraints	Description
room_id	UUID	PK	Unique identifier
room_type_id	UUID	FK → room_types.room_type_id, NOT NULL	Room type this room belongs to
room_number	VARCHAR(20)	NOT NULL	Displayed room number/name
status	room_status_enum	NOT NULL, default 'available'	available occupied dirty inspected out_of_order
		UNIQUE (room_type_id, room_number)	Prevents duplicate room numbers within the same room type

3.4 rate_plans

Column	Type	Constraints	Description
rate_plan_id	UUID	PK	Unique identifier
room_type_id	UUID	FK → room_types.room_type_id, NOT NULL	Applicable room type
name	VARCHAR(100)	NOT NULL	e.g. Non-refundable, Seasonal 2026
rate_multiplier	NUMERIC(4,2)	NOT NULL, default 1.00	Multiplier applied to base_rate
cancellation_policy	TEXT	NULL	Free-text cancellation terms
valid_from	DATE	NULL	Optional seasonal start date
valid_to	DATE	NULL	Optional seasonal end date

3.5 guests

Column	Type	Constraints	Description
guest_id	UUID	PK	Unique identifier
full_name	VARCHAR(150)	NOT NULL	Guest's full name
email	VARCHAR(150)	NULL	Contact email
phone	VARCHAR(30)	NULL	Contact phone
id_document_type	VARCHAR(30)	NULL	e.g. Passport, National ID
id_document_number	VARCHAR(60)	NULL, encrypted at rest	Identity document reference
created_at	TIMESTAMPTZ	NOT NULL, default now()	Record creation time

3.6 reservations

The central table of the system. A PostgreSQL exclusion constraint guarantees at the database level that no two reservations for the same room can have overlapping date ranges (see Section 4.1).

Column	Type	Constraints	Description
reservation_id	UUID	PK	Unique identifier
property_id	UUID	FK → properties.property_id, NOT NULL	Owning property
room_id	UUID	FK → rooms.room_id, NOT NULL	Assigned room

Column	Type	Constraints	Description
guest_id	UUID	FK → guests.guest_id, NOT NULL	Guest for this reservation
rate_plan_id	UUID	FK → rate_plans.rate_plan_id, NULL	Applied rate plan, if any
stay_range	TSTZRANGE	NOT NULL	Check-in/check-out as a single range value, used by the exclusion constraint
status	reservation_status_enum	NOT NULL, default 'confirmed'	pending confirmed checked_in checked_out cancelled no_show
created_by	UUID	FK → staff.staff_id, NULL	Staff member who created the booking (NULL for guest self-booking)
created_at	TIMESTAMPTZ	NOT NULL, default now()	Record creation time

3.7 folios

Column	Type	Constraints	Description
folio_id	UUID	PK	Unique identifier
reservation_id	UUID	FK → reservations.reservation_id, NOT NULL, UNIQUE	One folio per reservation
balance	NUMERIC(12,2)	NOT NULL, default 0	Current outstanding balance
status	folio_status_enum	NOT NULL, default 'open'	open closed

3.8 folio_line_items

Column	Type	Constraints	Description
item_id	UUID	PK	Unique identifier
folio_id	UUID	FK → folios.folio_id, NOT NULL	Owning folio
description	VARCHAR(200)	NOT NULL	e.g. Room charge, Minibar, Laundry
amount	NUMERIC(10,2)	NOT NULL	Charge amount (positive) or adjustment (negative)
posted_by	UUID	FK → staff.staff_id, NULL	Staff member who posted the charge

Column	Type	Constraints	Description
posted_at	TIMESTAMPTZ	NOT NULL, default now()	Timestamp of the charge

3.9 payments

Column	Type	Constraints	Description
payment_id	UUID	PK	Unique identifier
folio_id	UUID	FK → folios.folio_id, NOT NULL	Folio being paid
method	payment_method_enum	NOT NULL	card cash bank_transfer
amount	NUMERIC(10,2)	NOT NULL, CHECK (amount > 0)	Payment amount
gateway_reference	VARCHAR(100)	NULL	Transaction ID returned by the payment gateway
status	payment_status_enum	NOT NULL, default 'pending'	pending succeeded failed refunded
created_at	TIMESTAMPTZ	NOT NULL, default now()	Record creation time

3.10 staff

Column	Type	Constraints	Description
staff_id	UUID	PK	Unique identifier
property_id	UUID	FK → properties.property_id, NULL	NULL for an Owner with cross-property access
role_id	SMALLINT	FK → roles.role_id, NOT NULL	Assigned role
full_name	VARCHAR(150)	NOT NULL	Staff member's name
email	VARCHAR(150)	NOT NULL, UNIQUE	Login email
password_hash	VARCHAR(255)	NOT NULL	bcrypt/argon2 hash – never plaintext
is_active	BOOLEAN	NOT NULL, default true	Deactivated staff cannot log in

3.11 roles and permissions

A standard role/permission lookup pair supports fine-grained RBAC without hardcoding role checks throughout the codebase.

Column	Type	Constraints	Description
roles.role_id	SMALLINT	PK	e.g. 1=Owner, 2=Manager, 3=Front Desk, 4=Housekeeping, 5=Accountant
roles.name	VARCHAR(50)	NOT NULL, UNIQUE	Role display name
permissions.permission_id	SMALLINT	PK	e.g. reservations.create, folios.refund
role_permissions.role_id / permission_id	SMALLINT, SMALLINT	Composite PK, FKs	Many-to-many mapping of roles to permissions

3.12 audit_log

Column	Type	Constraints	Description
audit_id	UUID	PK	Unique identifier
staff_id	UUID	FK → staff.staff_id, NULL	Actor (NULL for system-initiated actions)
property_id	UUID	FK → properties.property_id, NULL	Related property
action	VARCHAR(100)	NOT NULL	e.g. reservation.cancel, folio.refund, staff.role_change
entity_type	VARCHAR(50)	NOT NULL	Table/entity affected
entity_id	UUID	NOT NULL	Identifier of the affected record
metadata	JSONB	NULL	Additional context (old/new values, reason)
created_at	TIMESTAMPTZ	NOT NULL, default now()	When the action occurred

4. Data Integrity and Constraints

4.1 Preventing Double-Booking

The reservations table uses the btree_gist PostgreSQL extension together with an EXCLUDE constraint on (room_id, stay_range), configured so that overlapping ranges are only rejected between reservations in an active status (confirmed, checked_in). This means the database itself will refuse to insert or update a reservation that would overlap an existing active reservation for the same room – this protection holds even under concurrent requests, which an application-level “check then insert” pattern cannot reliably guarantee.

Representative constraint (illustrative SQL):

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
```

```
ALTER TABLE reservations ADD CONSTRAINT no_overlapping_bookings
```

```
EXCLUDE USING gist (room_id WITH =, stay_range WITH &&)
WHERE (status IN ('confirmed', 'checked_in'));
```

4.2 Referential Integrity

- All foreign keys use ON DELETE RESTRICT by default to prevent accidental loss of historical booking/financial data.
- Soft-deletion (an is_active or deleted_at flag) is used for rooms, room types, and staff instead of hard deletes, preserving historical reservation and folio references.
- folios.balance is maintained via a database trigger that recalculates the balance whenever a folio_line_items or payments row is inserted, updated, or deleted, keeping it consistent without relying on application code to remember to update it.

4.3 Multi-Property Data Isolation

Every operationally significant table (rooms, reservations, folios, staff) carries a direct or indirect property_id. All API queries are automatically scoped by the authenticated user's assigned property/properties at the data-access layer, so it is not possible for a query to accidentally return another property's data even if application code has a bug elsewhere.

4.4 Indexing Strategy

Table	Index	Purpose
reservations	GIST (room_id, stay_range)	Supports the exclusion constraint and fast availability lookups
reservations	BTREE (property_id, status)	Fast filtering for dashboards and reports
folio_line_items	BTREE (folio_id)	Fast folio detail retrieval
payments	BTREE (folio_id, status)	Fast payment reconciliation
staff	UNIQUE BTREE (email)	Fast login lookups and uniqueness enforcement
audit_log	BTREE (property_id, created_at)	Efficient audit trail queries by date range

5. Backup and Recovery Strategy

- Automated daily full backups of the production database via the managed cloud provider.
- Point-in-time recovery enabled, allowing restoration to any point within the retention window (minimum 7 days).
- Backups stored in a separate region/availability zone from the primary database.
- Quarterly restore drills to verify backup integrity.

6. Migration Strategy

Schema changes are managed through versioned migration files (e.g., via Prisma Migrate), applied in a fixed order and tracked in a migrations history table. Migrations are reviewed and tested in the staging environment before being applied to production as an explicit, logged deployment step – never generated or applied automatically against production data.